

LLM-Based Automated Security Code Review Using Taint Analysis and Chain-of-Thought Reasoning

1 Executive Summary

LLMs can effectively perform automated security code review through taint analysis when deployed as neuro-symbolic hybrids—using LLMs for specification inference and contextual reasoning while relying on traditional program analysis engines for sound dataflow propagation. The most successful architecture is not LLM-only or SAST-only, but a layered integration where LLMs dynamically infer taint sources, sinks, and propagation rules that traditional tools cannot derive without manual configuration, then validate findings through chain-of-thought reasoning that simulates explicit data flow tracing.

The critical finding is the performance gap: IRIS detects 57.5% of vulnerabilities in CWE-Bench-Java versus only 22.5% by the best standalone SAST tool [1], while hybrid SAST+LLM frameworks achieve 91% false positive reduction compared to SAST alone [2]. Chain-of-thought prompting is essential to this performance—structured reasoning strategies like “Think & Verify” reduce ambiguous responses from 20.3% to 9.1% [3], and vulnerability-semantics-guided prompting achieves 553% higher F1 accuracy over baselines [4]. RAG architectures grounding analysis in CWE databases reach 0.86 accuracy [5], the highest among standalone prompting approaches.

For enterprise integration, the evidence supports a four-layer pipeline: deterministic SAST for baseline coverage, LLM-based triage for false positive elimination, semantic taint tracing on high-risk paths, and multi-agent deep analysis for critical releases. Context window limitations and inter-procedural analysis remain the primary constraints—Code Property Graph slicing reduces input size by 67–91% while preserving vulnerability context [6], and agent-based systems like RepoAudit demonstrate repository-scale analysis at approximately \$2.54 per project [7], making this economically viable for large portfolios today.

2 Introduction

The intersection of Large Language Models and static security analysis represents a paradigm shift in how organizations detect vulnerabilities at scale. Traditional taint analysis—the backbone of Static Application Security Testing (SAST)—has relied on manually curated rules defining sources, sinks, and propagation semantics for decades. While effective for known patterns, these tools suffer from rigid specifications that miss context-dependent vulnerabilities and produce substantial false positive rates when confronted with custom frameworks, dynamic language features, or novel API surfaces. The emergence of LLMs capable of semantic code reasoning introduces a fundamentally different approach: models that can infer taint specifications from code context, trace data flows through chain-of-thought decomposition, and validate findings against learned vulnerability patterns.

This report provides a technically deep analysis of how LLMs perform automated security code review through taint analysis and CoT reasoning, synthesizing research from 2024–2026. It examines the mechanisms by which LLMs replicate and enhance traditional dataflow tracking, surveys the rapidly evolving research landscape, presents quantitative comparisons against established SAST tools, and offers actionable guidance for integrating these capabilities into enterprise DevSecOps pipelines. The analysis draws on peer-reviewed publications from ICLR, OOP-SLA, ICSE, and ICML, alongside production deployments from GitHub, Snyk, Google DeepMind, and OpenAI.

3 Traditional Taint Analysis Foundations and LLM Enhancement

The Classical Taint Analysis Model. Traditional taint analysis operates on a well-defined three-step workflow: identifying taint sources (functions receiving external data such as `recv`, `getenv`, `fgets`), propagating taint labels along data dependencies including assignments, computations, and function calls, and checking whether tainted data reaches security-sensitive sinks without passing through appropriate sanitizers [8, 9]. The formal underpinning relies on lattice-based dataflow frameworks where taint states form a finite lattice and transfer functions model how each program statement transforms taint information. The IFDS framework (Reps, Horwitz, Sagiv) enables precise inter-procedural analysis by reducing taint tracking to graph reachability problems over exploded supergraphs, making it the foundation for tools like FlowDroid and modern CodeQL implementations [10].

Propagation rules specify which function arguments carry taint forward—configured via source-args and destination-args with indexed parameters—while sanitizer (filter) specifications remove taint from designated arguments [8]. The fundamental precision trade-off manifests as over-tainting versus under-tainting: over-tainting conservatively marks too much data as tainted, producing false positives, while under-tainting misses actual flows, causing false negatives. Critically, these errors accumulate during propagation across multiple operations, compounding imprecision in large codebases [11].

Where LLMs Transform the Model. The most significant limitation of traditional taint analysis is the manual

effort required to specify sources, sinks, and propagation rules for every library, framework, and custom API. As Theori’ s research demonstrates, tools like CodeQL require manual analysis of source/sink/sanitize APIs for third-party libraries, and missing specifications result in inaccurate findings [12]. LLMs address this bottleneck through three mechanisms. First, adaptive source/sink inference: the AdaTaint framework uses LLMs to dynamically infer candidate sources and sinks from project-specific APIs, commit history, and natural language documentation, overcoming the rigidity of manually curated rule sets and reducing false positives by approximately 44% while improving recall by about 11% compared to CodeQL and Joern baselines [13]. Second, automated propagation rule generation: LATTE demonstrates that LLMs can fully automate taint propagation rule customization for binary analysis, eliminating the need for human expertise and finding 37 new bugs in real-world firmware that traditional tools missed, with 10 assigned CVE numbers [9]. Third, semantic sanitizer understanding: LLMs demonstrate promising abilities at identifying vulnerability-related specifications and leveraging natural language information to understand code behavior, such as checking if code is properly sanitized [14].

4 Chain-of-Thought Reasoning for Data Flow Tracing

CoT as Structured Taint Propagation Simulation. Chain-of-thought prompting transforms LLM vulnerability detection from pattern matching into explicit dataflow reasoning. Rather than classifying code snippets as vulnerable or safe based on surface features, CoT forces the model to decompose analysis into discrete steps that mirror traditional taint propagation: (1) identify untrusted input sources, (2) trace variable assignments and function call chains, (3) evaluate whether transformations constitute valid sanitization, and (4) determine if tainted data reaches a security-sensitive sink. This structured decomposition maps directly onto the source→propagation→sink model of classical taint analysis but leverages the model’ s semantic understanding to handle cases where static rules fail.

Quantitative Impact of CoT Strategies. The Vulnerability-Semantics-guided Prompting (VSP) approach, which applies CoT-inspired reasoning to vulnerability analysis, achieved 553.3% higher F1 accuracy for vulnerability identification over baselines on CVE datasets [4]. This dramatic improvement stems from forcing the model to articulate its reasoning about data flow rather than relying on implicit pattern recognition. The multi-granular zero-shot framework proposed by researchers in 2025 supports analysis at function, file, and inter-function levels using four diverse prompt strategies: Baseline, Chain-of-Thought, Think, and Think & Verify [3]. The Think & Verify strategy proved particularly effective, reducing ambiguous responses from 20.3% to 9.1% while increasing accuracy—outperforming baseline, standard CoT, and simple “Think” prompts [3].

Practical CoT Decomposition for Taint Tracing. In practice, effective CoT prompts for taint analysis guide the model through a structured reasoning chain: first identifying all entry points where external data enters the function (HTTP parameters, file reads, database queries), then tracing each tainted variable through assignments, string concatenations, and function calls, explicitly noting whether each transformation preserves or removes taint, and finally evaluating whether the data reaches a sink (SQL query construction, command execution, file path resolution) in a tainted state. This approach is particularly powerful for detecting vulnerabilities in custom frameworks where traditional tools lack pre-configured rules, as the LLM can reason about sanitization semantics from function names, documentation comments, and implementation patterns.

5 Research Landscape: Key Papers 2024–2026

The past two years have produced a concentrated body of work demonstrating that LLM-augmented taint analysis substantially outperforms both standalone LLMs and traditional static analysis. The following table summarizes the most significant contributions:

Paper	Venue/Date	Methodology	Key Results
IRIS	ICLR 2025	Neuro-symbolic whole-repository taint analysis; LLMs infer taint specifications and perform contextual analysis	Detects 69/120 vulnerabilities in CWE-Bench-Java vs 27 by SOTA SAST; >80% false alarm reduction [1]
LLift	OOPSLA 2024	LLM-assisted static analysis agent for use-before-initialization bugs in Linux kernel	50% precision, ~100% recall on ~1000 potential UBI bugs; 13 previously unknown bugs found [15]

Paper	Venue/Date	Methodology	Key Results
GPTScan	ICSE 2024	GPT + static analysis for smart contract logic vulnerabilities; scenario/property matching	>90% precision for token contracts; >70% recall; 9 new vulnerabilities missed by auditors [16]
LATTE	arXiv, Jan 2025	First LLM-powered static binary taint analysis; automates propagation and inspection rules	37 new firmware bugs; 10 CVEs assigned; fully automated [17]
AdaTaint	2025	LLM-driven adaptive taint analysis; infers source/sink specs and filters spurious alerts	43.7% FP reduction; 11.2% recall improvement vs CodeQL/Joern [13]
BugLens	Apr 2025	LLM-based refinement of static analysis warnings for Linux kernel taint-style bugs	Precision improved 7-fold (0.10 to 0.72); 4 previously unreported vulnerabilities [18]
SemTaint	Jan 2025	Multi-agent LLM system extracting taint specifications combined with traditional static analysis	Extracts sources, sinks, call edges, and library flow summaries per package [19]

Table 1: Key Research Papers Combining LLMs with Taint Analysis (2024–2026)

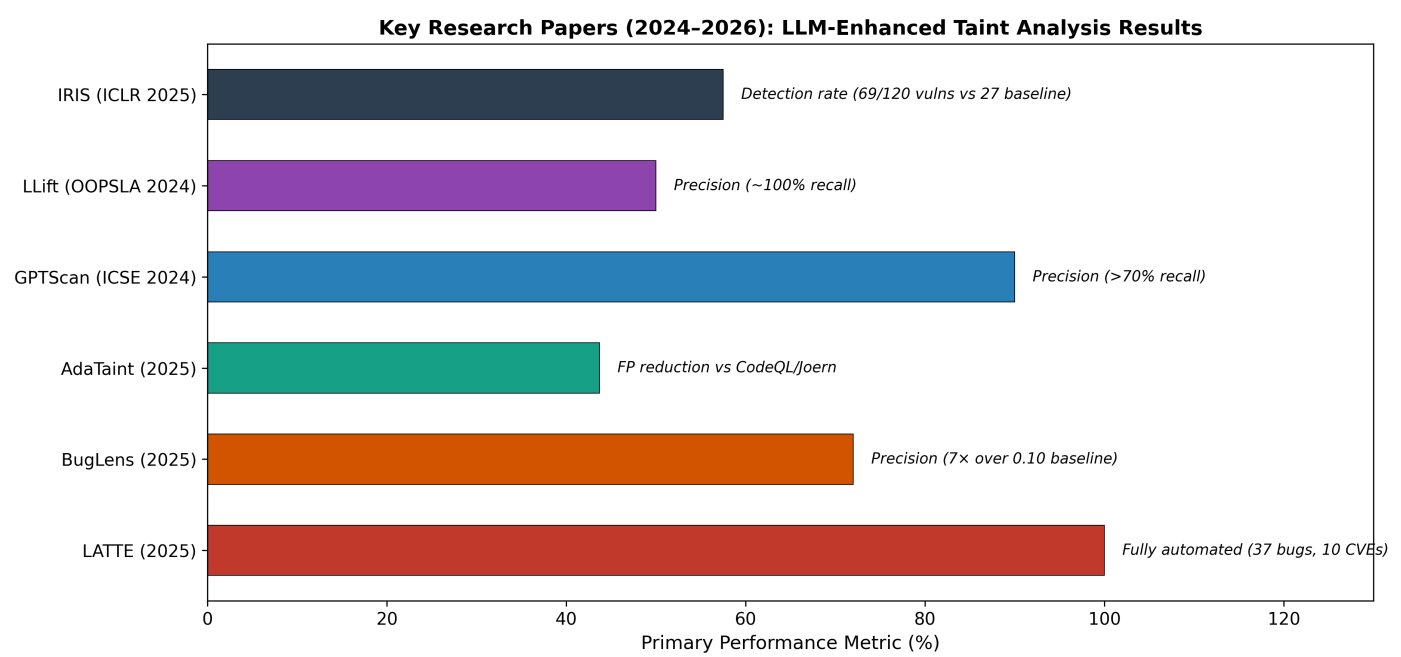


Figure 1: Research Paper Results

Chart Explanation: This chart visualizes the primary performance metric for each major research paper. IRIS achieves a 57.5% detection rate on CWE-Bench-Java (69/120 vulnerabilities) compared to only 22.5% by the best traditional SAST tool [1]. BugLens demonstrates a 7× precision improvement from a 0.10 baseline to 0.72 [18]. LATTE represents full automation with 37 new bugs discovered and 10 CVEs assigned [17].

Architectural Patterns Across Papers. A clear convergence emerges: the most successful approaches are neuro-symbolic hybrids that use LLMs for specification inference and contextual reasoning while relying on traditional program analysis for sound propagation. IRIS uses GPT-4 to infer taint specifications (sources and sinks) then feeds these into a classical taint analysis engine for whole-repository tracking [1]. SemTaint employs a multi-agent LLM system to extract taint specifications—including sources, sinks, call edges, and library flow summaries—tailored to each package, then combines these with traditional static program analysis [19]. This pattern acknowledges that LLMs excel at semantic understanding and specification inference but lack the formal guarantees needed for sound dataflow propagation across large codebases.

6 LLM vs Traditional SAST: Comparative Analysis

Quantitative Performance Gap. Empirical benchmarks consistently demonstrate that LLM-based approaches achieve substantially higher detection rates than traditional SAST tools, though with different error profiles. A 2025 study comparing 15 SAST tools with 12 LLMs found that SAST tools obtain low vulnerability detection rates with relatively low false positives, while LLMs can detect 90% to 100% of vulnerabilities but suffer from high false positives [20]. LLM-based scanners achieve higher mean F1 scores (0.797, 0.753, 0.750) than their static analysis counterparts (0.260, 0.386, 0.546), though LLMs exhibit the highest false-positive ratio and mislocate issues at line-level granularity due to tokenization artifacts [21].

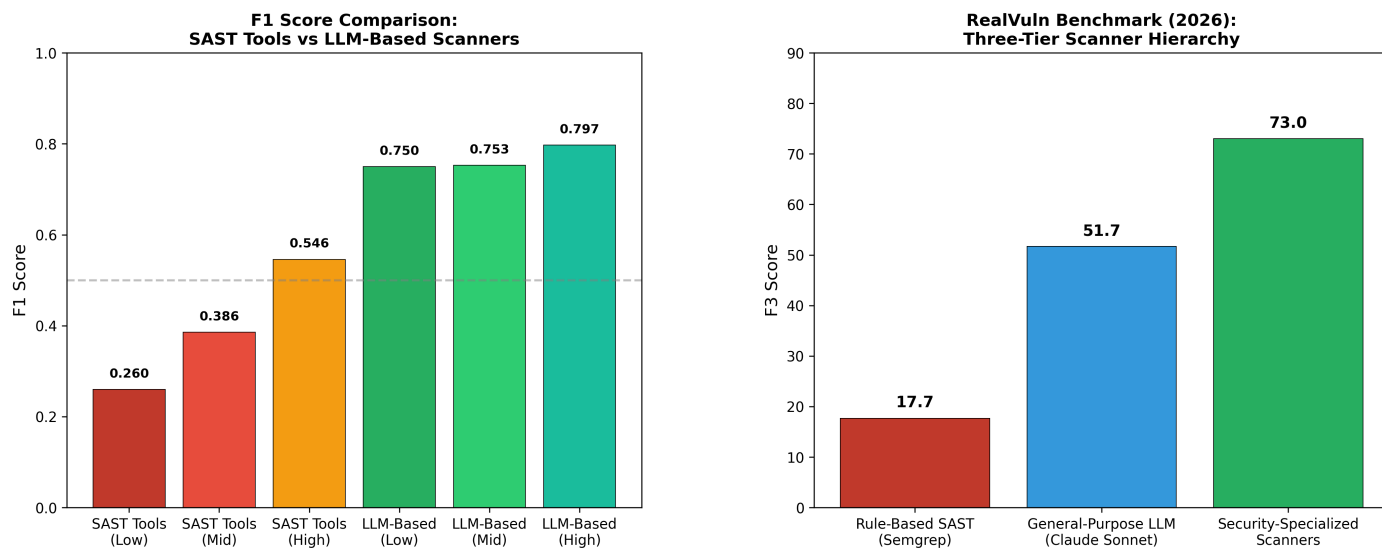


Figure 2: SAST vs LLM Performance Comparison

Chart Explanation: The left panel shows F1 score ranges for traditional SAST tools (0.260–0.546) versus LLM-based scanners (0.750–0.797), demonstrating a consistent performance advantage for LLM approaches [21]. The right panel presents the RealVuln benchmark’s three-tier hierarchy, where security-specialized scanners lead at F3=73.0, general-purpose LLMs like Claude Sonnet score 51.7, and rule-based SAST tools like Semgrep score only 17.7 [22].

The Three-Tier Hierarchy. The RealVuln benchmark (April 2026), which tested 15 scanners on 796 hand-labeled entries, revealed a clear three-tier hierarchy: security-specialized scanners lead with F3=73.0, followed by general-purpose LLMs like Claude Sonnet at F3=51.7, with rule-based SAST tools like Semgrep scoring lowest at F3=17.7 [22]. This hierarchy suggests that domain specialization—whether through fine-tuning, specialized prompting, or hybrid architectures—is essential for competitive security analysis performance.

Intra-procedural Strengths vs Inter-procedural Weaknesses. LLMs are significantly better at detecting vulnerabilities that typically only need intra-procedural reasoning, such as OS Command Injection and NULL Pointer Dereference, and report higher accuracies on these vulnerabilities than popular static analysis tools like CodeQL [23]. However, they struggle with inter-procedural analysis requiring multi-hop function calls across module boundaries—precisely the scenario where traditional SAST tools with call graph construction excel.

The Hybrid Advantage. The most compelling finding for enterprise adoption is that hybrid SAST+LLM frameworks achieve dramatic improvements over either approach alone. A novel SAST-LLM framework achieved a 91% reduction in false positives compared to standalone SAST tools by combining deterministic rule-based scanning with LLM contextual validation [2]. This architecture uses SAST for high-recall detection followed by LLM-based triage to eliminate false positives—a pattern directly applicable to enterprise pipelines where alert fatigue from false positives is the primary operational challenge.

7 Prompt Engineering and Architectural Patterns

Retrieval-Augmented Generation for Vulnerability Knowledge. RAG architectures that integrate external domain knowledge from vulnerability databases achieve the highest standalone accuracy. A comparative study found that RAG integrating the MITRE CWE database achieved the highest overall accuracy (0.86) and F1 score (0.85), outperforming both supervised fine-tuning and dual-agent frameworks [5]. This approach grounds LLM reasoning in authoritative vulnerability taxonomies, reducing hallucination risk and ensuring findings map to recognized weak-

ness categories.

Multi-Agent Architectures. The state of the art in LLM security analysis employs multi-agent systems with specialized roles. The Argus framework uses collaborative multi-agent workflows integrating RAG and ReAct reasoning to minimize hallucinations, performing supply chain analysis and full-chain vulnerability detection [24]. RAVEN implements four specialized modules: an Explorer agent for vulnerability identification, a RAG engine retrieving relevant knowledge from curated databases including Google Project Zero reports and CWE entries, an Analyst agent for impact and exploitation assessment, and a Reporter agent for structured report generation [25]. This decomposition mirrors how human security teams operate—with specialists for discovery, analysis, and reporting—and enables each agent to be optimized for its specific subtask.

Role-Based Persona Prompting. Assigning a specific security persona primes the model to apply domain-specific analysis lenses [26]. Effective personas include “senior security engineer specializing in taint analysis” or “application security architect reviewing for OWASP Top 10 vulnerabilities.” The key insight is that persona prompting activates different reasoning patterns within the model, causing it to prioritize security-relevant observations over general code quality concerns.

Cross-Model Prompt Evolution. A significant scalability challenge is that manual prompt engineering for the massive number of CWE categories is unscalable [27]. Cross-model prompt evolution addresses this by automatically evolving specialized prompts per vulnerability type, using iterative refinement across model generations to discover effective prompting strategies for each weakness class without human intervention.

Few-Shot Selection Strategy. Few-shot prompting effectiveness depends heavily on example selection; retrieval-augmented few-shot prompting that selects semantically similar vulnerable/safe code pairs outperforms random example selection [28]. The optimal approach retrieves examples that share structural similarity with the target code while demonstrating the specific vulnerability pattern under investigation.

8 Real-World Tools and Production Deployments

Enterprise Platform Landscape. The commercial ecosystem has rapidly incorporated LLM capabilities into security tooling. GitHub Copilot Autofix combines CodeQL detection with LLM-powered fix generation (GPT-4), covering more than 90% of alert types in JavaScript, TypeScript, Java, and Python with native pull request integration [29]. Snyk Code, built on the DeepCode acquisition, provides AI-powered SAST across 15+ languages with IDE plugins and CI/CD integration, claiming 84%+ MTTR reduction [30]. Amazon CodeGuru Reviewer uses ML trained on Amazon’s internal code reviews to identify critical defects in Java and Python, integrating with AWS CodePipeline [31].

More recent entrants include Google’s CodeMender (announced October 2025), an AI agent for both reactive patching and proactive code rewriting that builds on Big Sleep and OSS-Fuzz vulnerability discovery [32]. OpenAI’s Daybreak initiative (May 2026) combines frontier LLMs with Codex for secure-by-design development [33], while OpenAI’s Codex Security (May 2026) specifically combines agentic reasoning with automated validation to deliver high-confidence findings with reduced false positives [34].

Agent-Based Repository-Scale Analysis. RepoAudit represents the cutting edge of autonomous security analysis, using agent memory with data-flow validation to detect 40 true bugs across 15 real-world benchmark projects with a precision of 78.43%, requiring on average only 0.44 hours and \$2.54 per project [7]. It has detected 185 new bugs in high-profile projects, among which 174 have been confirmed or fixed—demonstrating that LLM-based analysis can operate at repository scale with economically viable cost structures.

CI/CD Integration Patterns. Production deployments follow a consistent integration pattern: SAST tools run on every commit for fast, deterministic baseline coverage, while LLM-based analysis triggers on pull requests or scheduled scans for deeper semantic review. The LLM layer typically operates as a triage filter—validating SAST findings to reduce false positives before they reach developers—or as a second-pass analyzer detecting semantic vulnerabilities that rule-based tools miss.

9 Challenges, Limitations, and Failure Modes

Context Window Constraints. Auditing code repositories with LLMs poses significant challenges because limited context windows degrade bug report quality, and analyzing large-scale repositories incurs substantial time and token costs [7]. Current evaluations that assess models on isolated functions or files ignore the broader execution and data-flow context essential for understanding real-world vulnerabilities, leading to both incorrect conclusions and flawed rationales [35]. Even with 128K+ token context windows, enterprise codebases with millions of lines of code cannot be analyzed holistically, requiring chunking strategies that may sever critical data flow paths.

Hallucination and Accuracy Degradation. State-of-the-art models report only approximately 54.5% balanced accuracy on vulnerability detection without sufficient context [36]. Accuracy drops by up to 45% on rigorously

verified datasets compared to performance on synthetic benchmarks [6], and LLM-generated explanations often contain hallucinations or lack semantic consistency with CWE categories [37]. Most false positives stem from reasoning errors rather than misclassification—the model constructs plausible but incorrect data flow narratives that appear convincing but do not reflect actual program behavior.

Inter-Procedural Analysis Limitations. Traditional LLM approaches focus on function-level analysis, overlooking crucial inter-procedural dependencies and broader program context [6]. Performance degrades notably for newly discovered vulnerabilities containing unfamiliar lexical or structural patterns, underscoring the need for continual adaptation [38]. This limitation is particularly acute for taint analysis, where vulnerability patterns frequently span multiple function calls, module boundaries, and even service boundaries in microservice architectures.

Emerging Solutions: Code Property Graphs and Hybrid Approaches. The LLMxCPG technique integrates Code Property Graphs with LLMs, reducing code size by 67–91% while preserving vulnerability-relevant context and achieving 15–40% F1-score improvements over state-of-the-art baselines [6]. This approach addresses the context window limitation by extracting only the semantically relevant code slices for LLM analysis. VulReaD uses security knowledge graphs to improve binary F1-scores by 8–10% and multi-class classification by approximately 30% Macro-F1 through better CWE-level reasoning [37]. The MoCQ neuro-symbolic framework uses LLMs to generate vulnerability detection patterns then executes them via static analysis on large codebases, combining LLM semantic understanding with the scalability of traditional pattern matching [39].

10 Enterprise Integration Strategy and Future Directions

Recommended Hybrid Pipeline Architecture. For enterprise adoption, the evidence strongly supports a layered architecture: (1) traditional SAST tools (CodeQL, Semgrep) provide fast, deterministic baseline scanning on every commit with high recall but known false positive rates; (2) LLM-based triage validates SAST findings, achieving up to 91% false positive reduction [2]; (3) LLM-augmented deep analysis performs semantic taint tracing on high-risk code paths identified by SAST or triggered by specific change patterns; (4) multi-agent systems with RAG provide comprehensive analysis for critical releases or high-risk components. This architecture preserves the speed and determinism of traditional tools while leveraging LLM semantic reasoning where it provides the greatest marginal value.

Reasoning Models and Next-Generation Capabilities. OpenAI’s o1-preview achieves a 92.85% success rate on automated vulnerability exploitation benchmarks [40], demonstrating that extended inference-time reasoning dramatically improves security analysis capabilities. As reasoning models with longer chain-of-thought capabilities become available, the gap between LLM-based and traditional analysis will likely widen further—particularly for complex inter-procedural vulnerabilities that require multi-step reasoning about data flow across module boundaries.

Hybrid LLM + Symbolic Execution. Multiple systems now combine LLMs with symbolic or concolic execution, where LLMs guide path constraint selection and prioritization while symbolic execution provides formal verification [41]. This combination addresses the fundamental limitation of LLM-only approaches—lack of formal soundness guarantees—while leveraging LLM semantic understanding to overcome the path explosion problem that limits traditional symbolic execution.

Actionable Integration Recommendations. For organizations building LLM-augmented security pipelines, the evidence supports several concrete strategies: deploy RAG with CWE/MITRE databases as the foundation for vulnerability knowledge grounding (0.86 accuracy) [5]; implement Think & Verify structured prompting to minimize ambiguous findings [3]; use CPG-based code slicing to manage context window constraints while preserving vulnerability-relevant context [6]; adopt multi-agent architectures with specialized roles for discovery, analysis, and reporting [24, 25]; and establish continuous prompt evolution pipelines that automatically adapt to new vulnerability classes [27]. The economics are favorable—RepoAudit demonstrates that comprehensive repository-scale analysis is achievable at approximately \$2.54 per project [7]—making LLM-augmented security review viable even for organizations with large portfolios.

2026+ Trajectory. The field is converging toward fully autonomous security agents that combine specification inference, formal analysis, and semantic reasoning in unified frameworks. The progression from LATTE’s automated rule generation [17] through IRIS’s neuro-symbolic repository analysis [1] to RepoAudit’s autonomous bug detection [7] traces a clear trajectory toward systems that can independently discover, validate, and remediate vulnerabilities with minimal human oversight—transforming the security architect’s role from manual reviewer to orchestrator of AI-driven security assurance pipelines.

11 References

1. <https://arxiv.org/abs/2405.17238>

2. <https://www.infoworld.com/article/4093079/how-pairing-sast-with-ai-dramatically-reduces-false-positives-in-code-security.html>
3. <https://arxiv.org/abs/2503.17885>
4. <https://arxiv.org/abs/2402.17230>
5. <https://arxiv.org/html/2601.00254>
6. <https://arxiv.org/html/2507.16585v1>
7. <https://icml.cc/virtual/2025/poster/45170>
8. <https://releases.llvm.org/16.0.0/tools/clang/docs/analyzer/user-docs/TaintAnalysisConfiguration.html>
9. <https://arxiv.org/html/2310.08275v2>
10. <https://arxiv.org/abs/2103.16240>
11. <https://ar5iv.labs.arxiv.org/html/1907.03756>
12. <https://theori.io/blog/offensive-security-with-large-language-models-2-48547>
13. <https://arxiv.org/html/2511.04023>
14. <https://arxiv.org/abs/2311.16169>
15. <https://dl.acm.org/doi/10.1145/3649828>
16. <https://gptscan.github.io/>
17. <https://arxiv.org/abs/2310.08275v2>
18. <https://arxiv.org/abs/2504.11711>
19. <https://arxiv.org/abs/2601.10865>
20. <https://arxiv.org/abs/2407.16235>
21. <https://arxiv.org/html/2508.04448v1>
22. <https://arxiv.org/abs/2604.13764>
23. <https://arxiv.org/html/2311.16169v3>
24. <https://api.emergentmind.com/papers/2604.06633>
25. <https://arxiv.org/abs/2604.17948>
26. <https://crashoverride.com/blog/prompting-llm-security-reviews>
27. <https://arxiv.org/html/2601.18847v1>
28. <https://arxiv.org/html/2512.04106>
29. <https://github.blog/2024-03-20-found-means-fixed-introducing-code-scanning-autofix-powered-by-github-copilot-and-codeql/>
30. <https:// snyk.io/solutions/secure-ai-generated-code/>
31. <https://aws.amazon.com/codeguru/reviewer/faqs/>
32. <https://deepmind.google/blog/introducing-codemender-an-ai-agent-for-code-security/>
33. <https://www.infosecurity-magazine.com/news/openai-daybreak-secure-by-design/>
34. <https://openai.com/index/codex-security-now-in-research-preview/>
35. <https://arxiv.org/html/2504.13474>
36. <https://arxiv.org/abs/2403.17218>
37. <https://arxiv.org/html/2602.10787>
38. <https://arxiv.org/abs/2404.00287>
39. <https://arxiv.org/abs/2504.16057>
40. <https://arxiv.org/abs/2410.21939>
41. <https://www.emergentmind.com/topics/concolic-execution-with-large-language-models>